- *Payload*—A JSON object (called *Claims Set*) containing the claims conveyed by the token. The JWT specification defines some standard claim names, but you can also define your own. A decoded payload looks like the following:

```
{
  "iss": "https://sso.polarbookshop.com",     ❶
  "sub": "isabelle",                          ❷
  "exp": 1626439022                           ❸
}
```

❶ The entity that issued the JWT (the issuer)

❷ The entity that is the subject of the JWT (the end-user)

❸ When the JWT expires (a timestamp)

- *Signature*—The signature of the JWT, ensuring that the claims have not been tampered with. A prerequisite of using a JWS structure is that we trust the entity issuing the token (the *issuer*), and we have a way to check its validity.

When a JWT requires integrity and confidentiality, it's first signed as a JWS and then encrypted with JSON Web Encryption (JWE). In this book, we'll use only JWS.

NOTE If you're interested in learning more about JWT and its related aspects, you can refer to the IETF standard specifications. JSON Web Token (JWT) is documented in RFC 7519 (https://tools.ietf.org/html/rfc7519), JSON Web Signature (JWS) is described in RFC 7515 (https://tools.ietf.org/html/rfc7515), and JSON Web Encryption (JWE) is presented in RFC 7516 (https://tools.ietf.org/html/rfc7516). You might also be interested in JSON Web Algorithms (JWA), which defines the available cryptographic operations for JWTs and is detailed in RFC 7518 (https://tools.ietf.org/html rfc7518).

In the case of Polar Bookshop, Edge Service can delegate the authentication step to Keycloak. After authenticating the user successfully, Keycloak will send a JWT to Edge Service with information about the newly authenticated user (ID Token). Edge Service will validate the JWT through its signature